

Forgetful BFS

Miško tried to implement a breadth-first search (BFS) but he totally forgot that you should also check whether you've already visited the nodes. His implementation was simply the following:

```
for each node x: visited[x] = false

steps = 0
queue.push(start)
visited[start] = true

while not queue.empty():
    steps += 1
    current = queue.pop_front()
    for next in neighbors(current):
        visited[next] = true
        queue.push(next)
```

In the code, `neighbors(current)` is the list of all the neighbors of the given node, sorted by their number from smallest to largest.

Task

Assuming the queue never overflows, how long does it take to visit all reachable nodes in the graph? More precisely, what is the value of `steps` at the moment when the array `visited` changes for the last time?

Input format

The first line of each input file contains the number t of test cases. The specified number of test cases follows, one after another.

Each test case starts with a single line with two integers n and m : the number of nodes and the number of edges between them. The nodes are numbered from 0 to $n - 1$. The starting node for the BFS is always node 0.

The rest of the test case contains m lines, each containing the numbers of two nodes that are connected by an edge. The edges are undirected and the graph is simple (no self-loops or multiple edges).

Each test case has $0 \leq m \leq n(n - 1)/2$.

Output format

For each test case output a single line with the correct value `steps`. As this value can be astronomically large, output it modulo $10^9 + 7$.

Subproblem F1 (10 points, public)

Input file: [F1.in \(/static/online2026/F1.11f3e6bff722b6a19c4b319b3ede0210.in\)](/static/online2026/F1.11f3e6bff722b6a19c4b319b3ede0210.in).

Constraints: $t = 100$ and in each test case $1 \leq n \leq 16$.

Subproblem F2 (35 points, public)

Input file: [F2.in \(/static/online2026/F2.a4a75112b4b4c264c6f3d11f6dee4bed.in\)](/static/online2026/F2.a4a75112b4b4c264c6f3d11f6dee4bed.in).

Constraints: $t = 100$ and in each test case $1 \leq n \leq 50$.

Subproblem F3 (55 points, secret)

Input file: [F3.in \(/static/online2026/F3.e163215dd20a11100b8471f4dba4cd66.in\)](/static/online2026/F3.e163215dd20a11100b8471f4dba4cd66.in).

Constraints: $t = 100$ and in each test case $1 \leq n \leq 5\,000$ and $m \leq 50\,000$.

Example

input	output
<pre> 3 5 3 0 1 2 4 2 1 5 4 0 1 0 2 0 3 0 4 6 8 0 1 0 2 0 3 1 2 1 3 2 3 3 5 4 5 </pre>	<pre> 4 1 14 </pre>

Test case 1 looks as follows: 0 -- 1 -- 2 -- 4 3.

The variable `current` has values 0, 1, 0, 2. During the processing of `current == 2` we set `visited[4]` to `true`, and that's the last change. (Node 3 will never be visited.)

Test case 2 is a star with 0 in the center. All nodes are marked as visited once 0 is popped from the queue and processed.

Test case 3: the nodes are processed in the following order: 0, 1, 2, 3, 0, 2, 3, 0, 1, 3, 0, 1, 2, 5.